BEIHANG UNIVERSITY
北京航空航天大學

离散数学-逻辑语言

主要内容

- 2.1 命题合式公式语义
- 2.2 推论式与等价式的语义
- 2.3 变换合式公式的语义
- 2.4 命题公式范式
- 2.5 等式演算
- 2.6 完全集

命题逻辑公式的真值

- 对于一个命题逻辑合式公式，其逻辑真值是什么？
  - 什么情况下为真？
  - 什么情况下为假？
- 一个合式公式与逻辑真值之间的对应关系。

指派函数

- **定义 2.1.1 指派函数**（简称**指派**）是一种从合式公式到集合$\{0,1\}$的函数，记为$\boldsymbol{\sigma}$。
- $\boldsymbol{\sigma}$: $\boldsymbol{Q} \to \{\mathbf{0},\ \mathbf{1}\}$
- **定义 2.1.2** 设$\boldsymbol{Q}$是命题变元，$\boldsymbol{\sigma}$是指派，则$\boldsymbol{Q}$的语义是指定$\boldsymbol{Q}$的真值（表示$\boldsymbol{\sigma}$赋给命题变元$\boldsymbol{Q}$的真值），即$\boldsymbol{\sigma}(\boldsymbol{Q}) = \boldsymbol{Q}^{\boldsymbol{\sigma}}$，其中$\boldsymbol{Q}^{\boldsymbol{\sigma}} \in \{\mathbf{0},\mathbf{1}\}$。

合式公式语义

- **定义2.1.3** 设$\boldsymbol{Q}$，$\boldsymbol{R}$是命题公式，$\boldsymbol{\sigma}$是指派，合式公式联接词的语义指定为对应的逻辑运算。
  - (1) $\boldsymbol{\sigma(0) = 0}$，$\boldsymbol{\sigma(1) = 1}$
  - (2) $(\neg \boldsymbol{Q})$是合式公式，$\boldsymbol{\sigma(\neg Q) = \neg \sigma(Q)}$
  - (3) $(\boldsymbol{Q} \wedge \boldsymbol{R})$是合式公式，$\boldsymbol{\sigma(Q \wedge R) = \sigma(Q) \wedge \sigma(R)}$
  - (4) $(\boldsymbol{Q} \vee \boldsymbol{R})$是合式公式，$\boldsymbol{\sigma(Q \vee R) = \sigma(Q) \vee \sigma(R)}$
  - (5) $(\boldsymbol{Q} \rightarrow \boldsymbol{R})$是合式公式，$\boldsymbol{\sigma(Q \rightarrow R) = \sigma(Q) \rightarrow \sigma(R)}$
  - (6) $(\boldsymbol{Q} \leftrightarrow \boldsymbol{R})$是合式公式，$\boldsymbol{\sigma(Q \leftrightarrow R) = \sigma(Q) \leftrightarrow \sigma(R)}$
  - (7) $(\boldsymbol{Q} \oplus \boldsymbol{R})$是合式公式，$\boldsymbol{\sigma(Q \oplus R) = \sigma(Q) \oplus \sigma(R)}$

合式公式语义

- 合式公式的联结词¬，∧，∨，→，↔，⊕的语义解释为对应的逻辑域集合上的运算
- 符号**0**和**1**的语义解释为逻辑真值集合上的对象，即$\{\mathbf{0},\mathbf{1}\}$。
- 指派将命题变元指定为一个真值，将合式公式中的联结词变换为对应的逻辑运算，任意命题逻辑合式公式，经过指派求值运算，可得到对应的一个真值

推论式和等价式语义

- **定义 2.1.4 （推论式）**

若Q, R是合式公式，$\sigma(Q \vDash R) = \sigma(Q) \vDash \sigma(R)$

- **定义 2.1.5 （等价式）**

若Q, R是合式公式，

$$\sigma(Q \Leftrightarrow R) = \sigma(Q) \Leftrightarrow \sigma(R)$$

- 推论式和等价式的$\vDash$和$\Leftrightarrow$解释为逻辑域上相应的逻辑关系判断，符号$\Leftrightarrow$有时也表示为$=$

公式的可满足性和有效性

- **定义 2.1.6** 设$\boldsymbol{Q}$是公式。
- (1)如果真值指派$\boldsymbol{\sigma}$使得$\boldsymbol{\sigma}(\boldsymbol{Q}) = \mathbf{1}$，则称$\boldsymbol{\sigma}$**满足**$\boldsymbol{Q}$。
- (2)如果每个真值指派都满足$\boldsymbol{Q}$，则称$\boldsymbol{Q}$为**有效式**，或称为**永真式**，也称为**重言式**。
- (3)如果每个真值指派都不满足$\boldsymbol{Q}$，则称Q为**永假式**，也称为**矛盾式**，**不可满足式**。
- (4)如果至少有一个真值指派满足$\boldsymbol{Q}$，则称Q为**可满足式**。

Python

- {True，False}
- ∧:p&q
- ∨:p|q
- ¬:(not p)
- ⊕:p^q
- →:(not p) |q
- ↔:((not p) |q)&((not q) |p)

  p==q

求合式公式合、推论式与等价式语义语义方法

- 指派函数求真值法
- 真值表求真值法
- Python程序求真值法

命题逻辑公式语义求值

- **例 2.1.1** 求合式公式$s = P \to (Q \to R)$的真值。

设指派函数$P^\sigma = 1, Q^\sigma = 1, R^\sigma = 0$	设指派函数$P^\sigma = 0, Q^\sigma = 1, R^\sigma = 0$
$\sigma(P \to (Q \to R))$ $= \sigma(P) \to \sigma(Q \to R)$ $= \sigma(P) \to (\sigma(Q) \to \sigma(R))$ $= \sigma(P) \to (\sigma(Q) \to \sigma(R))$ $= P^\sigma \to (Q^\sigma \to R^\sigma)$ $= 1 \to (1 \to 0)$ $= 0$	$\sigma(P \to (Q \to R))$ $= \sigma(P) \to \sigma(Q \to R)$ $= \sigma(P) \to (\sigma(Q) \to \sigma(R))$ $= P^\sigma \to (Q^\sigma \to R^\sigma)$ $= 0 \to (1 \to 0)$ $= 1$

在不同的指派情况下，同一个合式公式的真值可能不同。

简化计算

- **例 2.1.2**：求合式公式$\neg(p \wedge q) \rightarrow \neg p \vee \neg q$的真值
- 设$p^{\sigma} = 0$或$q^{\sigma} = 0$

$\sigma(\neg(p \wedge q) \rightarrow \neg p \vee \neg q)$

$= \neg(\sigma(p) \wedge \sigma(q)) \rightarrow \neg\sigma(p) \vee \neg\sigma(q)$

$= 1 \rightarrow 1$

$= 1$

- 设$p^{\sigma} = 1$且$q^{\sigma} = 1$

$\sigma(\neg(p \wedge q) \rightarrow \neg p \vee \neg q)$

$= \neg(\sigma(p) \wedge \sigma(q)) \rightarrow \neg\sigma(p) \vee \neg\sigma(q)$

$= \neg 1 \rightarrow 0 \vee 0$

$= 1$

无论如何指派，该公式的真值总为1。

真值表

- 真值表由逻辑变量每种取值组合以及相对应的唯一值列表组成。
  - 每个逻辑变量均有0、1两种取值
  - 按二进制数递增方式排列起来
  - 每个逻辑公式为一列
  - 将对应的逻辑函数值写相应位置上
- 真值表用来定义联结词
- 真值表用来验证合式公式

真值表				
p	q	r	f_0	f_1
0	0	0	0	1
0	0	1	1	1
0	1	0	0	0
0	1	1	1	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

真值表求公式真值

- **例 2.1.3** 求公式$\neg(q \wedge r) \leftrightarrow (\neg q \vee \neg r)$真值
- 首先将公式$\neg(q \wedge r) \leftrightarrow (\neg q \vee \neg r)$按公式产生结构逐次分解成$q \wedge r, \neg(q \wedge r), \neg q, \neg r, \neg q \vee \neg r$子公式，而后分别求其值，最后求得公式$\neg(q \wedge r) \leftrightarrow (\neg q \vee \neg r)$真值。

q	r	$q \wedge r$	$\neg(q \wedge r)$	$\neg q$	$\neg r$	$\neg q \vee \neg r$	$\neg(q \wedge r) \leftrightarrow (\neg q \vee \neg r)$
0	0	0	1	1	1	1	1
0	1	0	1	1	0	1	1
1	0	0	1	0	1	1	1
1	1	1	0	0	0	0	1

- 真值表求合式公式逻辑值有容易、直观的优点。
- 当命题变量较多时，命题变量真值有组合数大，公式复杂，难以计算等缺点。

Python程序求真值表

```
def print01(truthtable):
    for tv in truthtable:
        if(tv):
            print ("1",  end = "  ")
        else:
            print ("0",  end = "  ")
    print("")
    return
```

```
def truthtable2(s):
    w='Q  R  '+s
    print("%r"%w)
    truth={True,  False}
    for Q in truth:
        for R in truth:
            f=eval(s)
            t=[Q,  R]+[f]
            print01(t)
    return
```

```
import dmath.logic as dml
s='(not(Q&R))==((not Q)|(not R))'
ml.truthtable2(s)
```

```
'Q  R  (not(Q&R))==((not Q)|(not R))'
0  0  1
0  1  1
1  0  1
1  1  1
```

Python程序求真值表（续）

```
def truthtable3(s):
    w='P  Q  R  '+s
    print("%r"%w)
    truth={True，False}
    for P in truth:
        for Q in truth:
            for R in truth:
                f=eval(s)
                t=[P，Q，R]+[f]
                print01(t)
    return
```

Python程序求真值表

- 求公式$\neg(p \wedge q) \leftrightarrow (\neg p \vee \neg q)$值。
- 设计Python程序实现真值表求值

```
def truthvalue( ):
  print("p  q  f0 f1 f2 f3 f4 f5")
  truth={True,False}
  tv=True
  for q in truth:
    for r in truth:
      f0=q&r
      f1=(not f0)
      f2=(not q)
      f3=(not r)
      f4=f2|f3
      f5=((not f1)|f4)&((not f4) | f1)
      print((q,r,f0,f1,f2,f3,f4,f5))
  return
```

```
def main():
  truthvalue( )
  return
```

```
main()
p  q  f0 f1 f2 f3 f4 f5
0  0  0  1  1  1  1  1
0  1  0  1  1  0  1  1
1  0  0  1  0  1  1  1
1  1  1  0  0  0  0  1
```

Python程序赋值函数

- 命题逻辑公式的语义都能够通过程序实现
- 设计Python程序求命题逻辑公式的真值。
- [Q，R]=[0，0]作为赋值。
- **例 2.1.2** 求$(\neg(\boldsymbol{Q} \wedge \boldsymbol{R})) \rightarrow ((\neg \boldsymbol{Q}) \vee (\neg \boldsymbol{R}))$

赋值$\boldsymbol{\sigma}$	真值表：任意赋值$\boldsymbol{\sigma}$
s='(¬(Q∧R))→((¬Q)∨(¬R))' s='(not(not(Q&R)))|((not Q)|(not R))' [Q,R]=[0,0] tv1=eval(s) [Q,R]=[1,0] tv2=eval(s) print(tv1) print(tv2)	import dmath.logic as dml s='((not(not(Q&R)))|((not Q)|(not R)))' dml.truthtable2(s) 'Q R ((not(not(Q&R)))|((not Q)|(not R)))' 0 0 1 0 1 1 1 0 1 1 1 1
True True	

谢谢